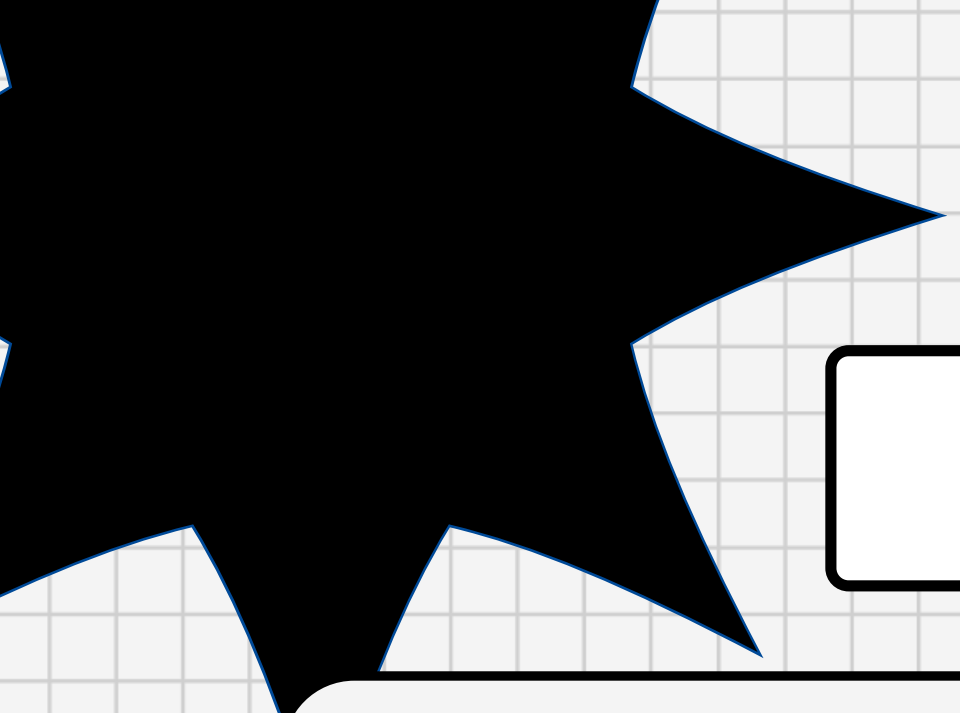
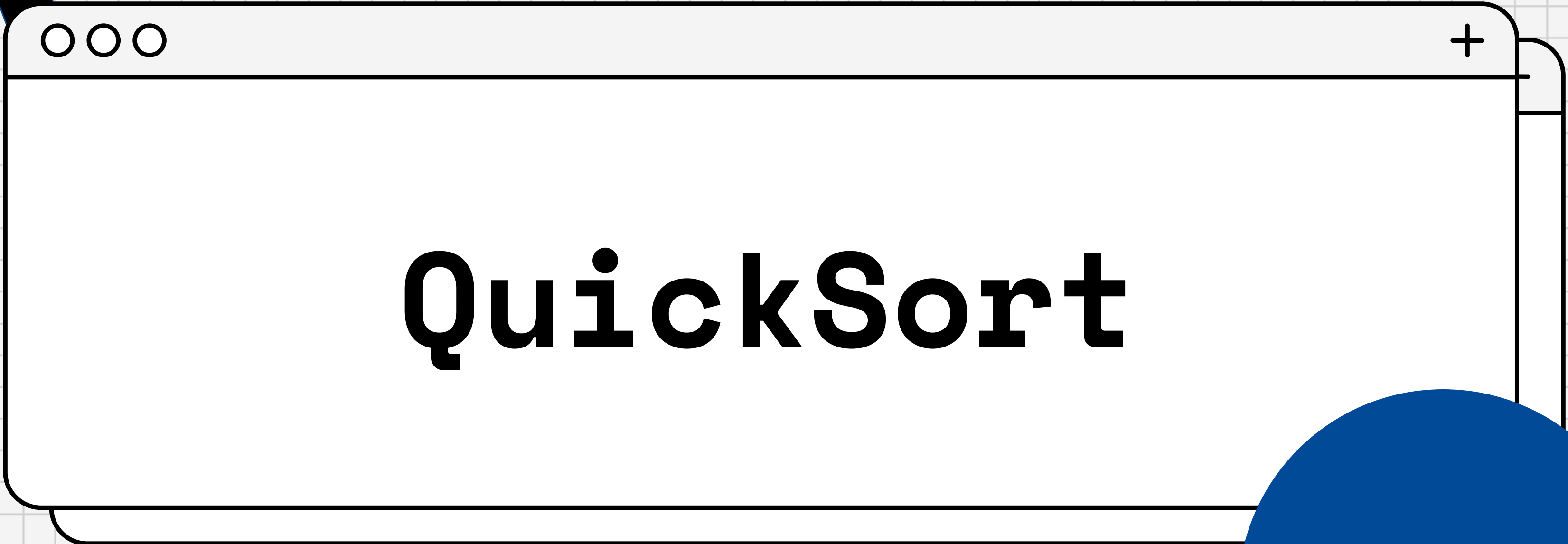
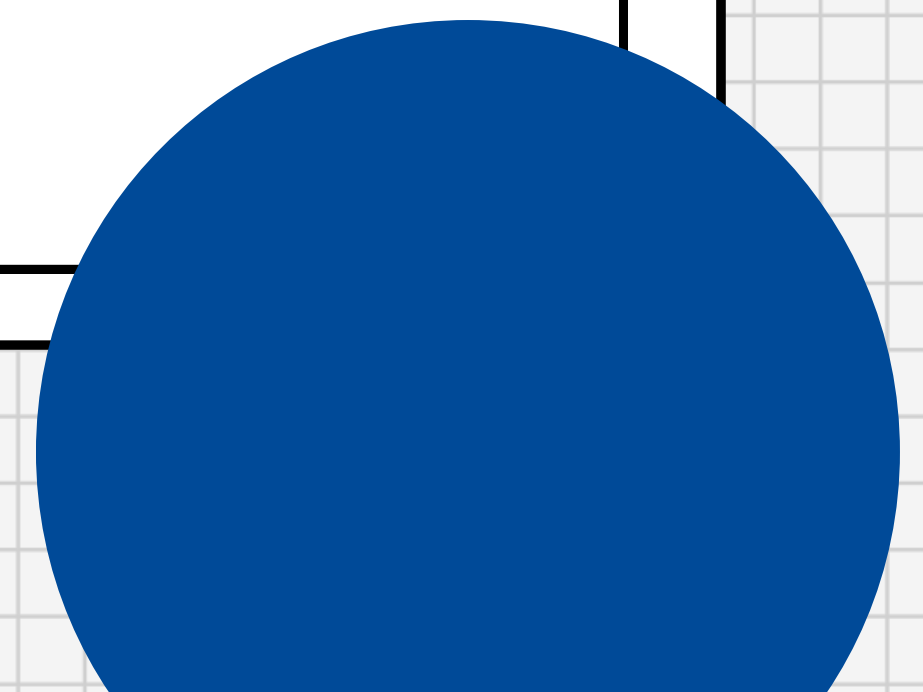




Ayudantía N° 5



QuickSort



Repaso

Quick Sort

- Elige el pivote
- Coloca al pivote en su posición correcta en el arreglo ordenado
- Pone los elementos menores que él a su izquierda y los mayores a su derecha

```
Partition ( $A, i, f$ ):  
1   $x \leftarrow$  índice aleatorio en  $\{i, \dots, f\}$   
2   $p \leftarrow A[x]$   
3   $A[x] \rightleftharpoons A[f]$   
4   $j \leftarrow i$   
5  for  $k = i \dots f - 1$  :  
6      if  $A[k] < p$  :  
7           $A[j] \rightleftharpoons A[k]$   
8           $j \leftarrow j + 1$   
9   $A[j] \rightleftharpoons A[f]$   
10 return  $j$ 
```

Quick Sort

Pseudocódigo 🔍

Elegimos un número aleatorio como el índice del pivote

Guardamos el pivote

Queremos corregir el arreglo para que quede:
[x < pivote, pivote, x > pivote]

Devolvemos el pivote a su lugar correcto

Partition(Arreglo, inicio, fin)

índice_pivote $\leftarrow$ índice aleatorio en [inicio, fin]

valor_pivote $\leftarrow$ Arreglo[índice_pivote]

Arreglo[índice_pivote] $\leftrightarrow$ Arreglo[fin]

índice_menor_que_el_pivote $\leftarrow$ inicio

for *índice_correccion* = *inicio* ... *fin* - 1 **do**

 valor_correccion $\leftarrow$ Arreglo[índice_correccion]

if *valor_correccion* < *valor_pivote* **then**

 Arreglo[índice_menor_que_el_pivote] $\leftrightarrow$ Arreglo[índice_correccion]

 índice_menor_que_el_pivote++

Arreglo[índice_menor_que_el_pivote] $\leftrightarrow$ Arreglo[fin]

Nota: Mucho texto, pero texto útil

Quick Sort



Inplace?



Input: secuencia A y
dos enteros i, f

Output: Nada...?

```
QuickSort ( $A, i, f$ ):  
1   if  $i \leq f$  :  
2        $p \leftarrow \text{Partition}(A, i, f)$   
3       Quicksort( $A, i, p - 1$ )  
4       Quicksort( $A, p + 1, f$ )
```

Quick Sort

```
QuickSort (A, i, f):  
1  if  $i \leq f$  :  
2       $p \leftarrow \text{Partition}(A, i, f)$   
3      Quicksort(A, i,  $p - 1$ )  
4      Quicksort(A,  $p + 1$ , f)
```

```
Partition (A, i, f):  
1   $x \leftarrow$  índice aleatorio en  $\{i, \dots, f\}$   
2   $p \leftarrow A[x]$   
3   $A[x] \rightleftharpoons A[f]$   
4   $j \leftarrow i$   
5  for  $k = i \dots f - 1$  :  
6      if  $A[k] < p$  :  
7           $A[j] \rightleftharpoons A[k]$   
8           $j \leftarrow j + 1$   
9   $A[j] \rightleftharpoons A[f]$   
10 return  $j$ 
```

Complejidad

Mejor y caso promedio

$O(n \log(n))$

Peor caso

$O(n^2)$

Memoria adicional

$O(1)$

quick sort

A =

4	2	1	3
---	---	---	---

`QuickSort(A, 0, 3)`

quick sort



A =

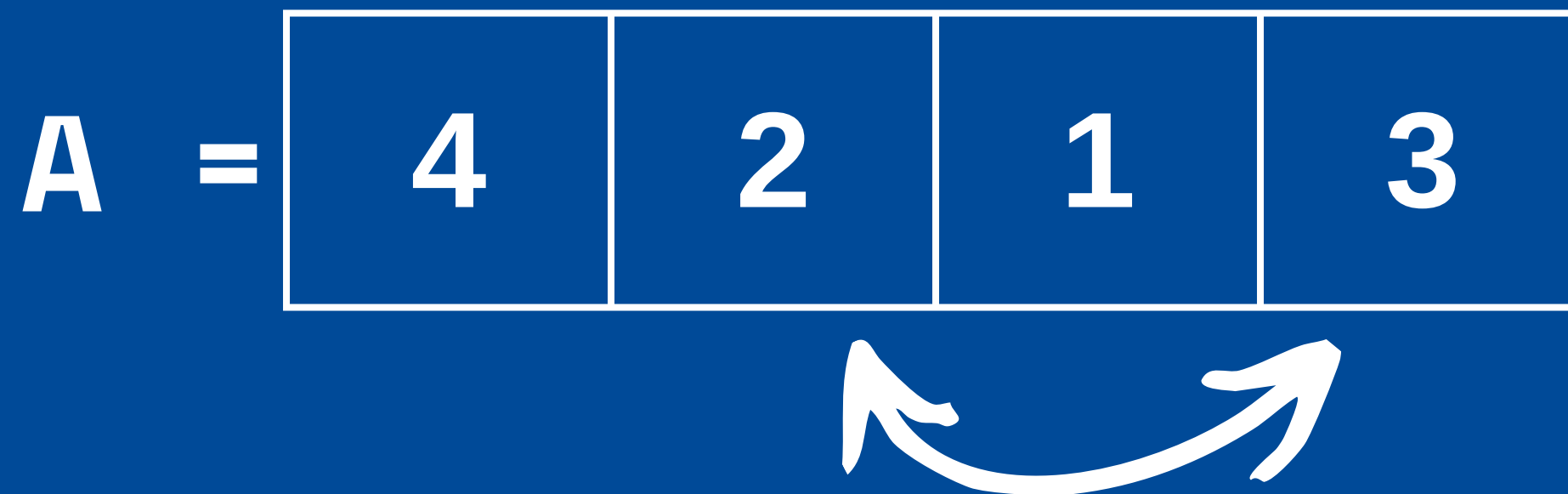
4	2	1	3
----------	----------	----------	----------

Elegimos un pivote aleatorio

$$x = 1$$

$$p = A[1] = 2$$

quick sort



Movemos el pivote al final

quick sort

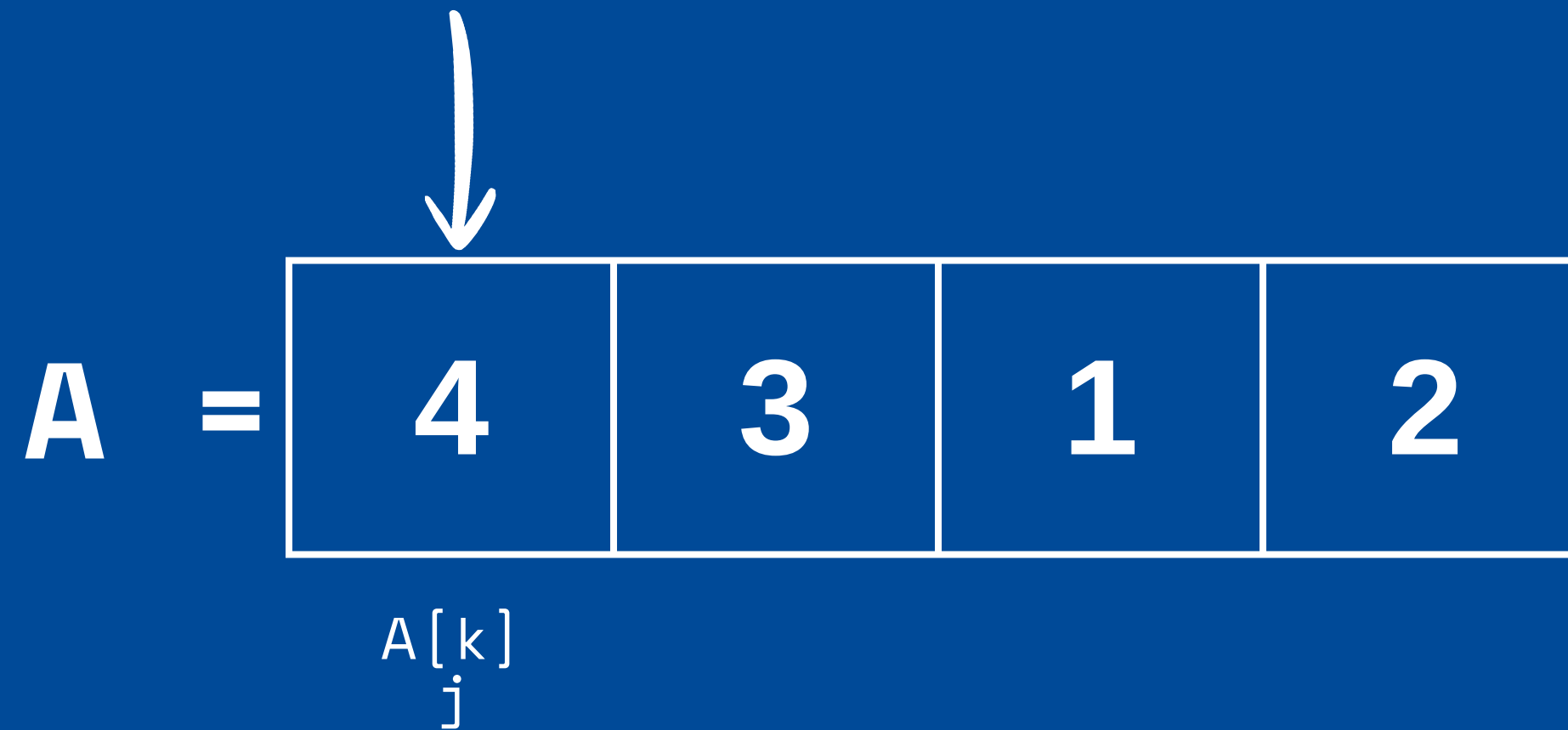
A =

4	3	1	2
---	---	---	---

$j = i = 0$

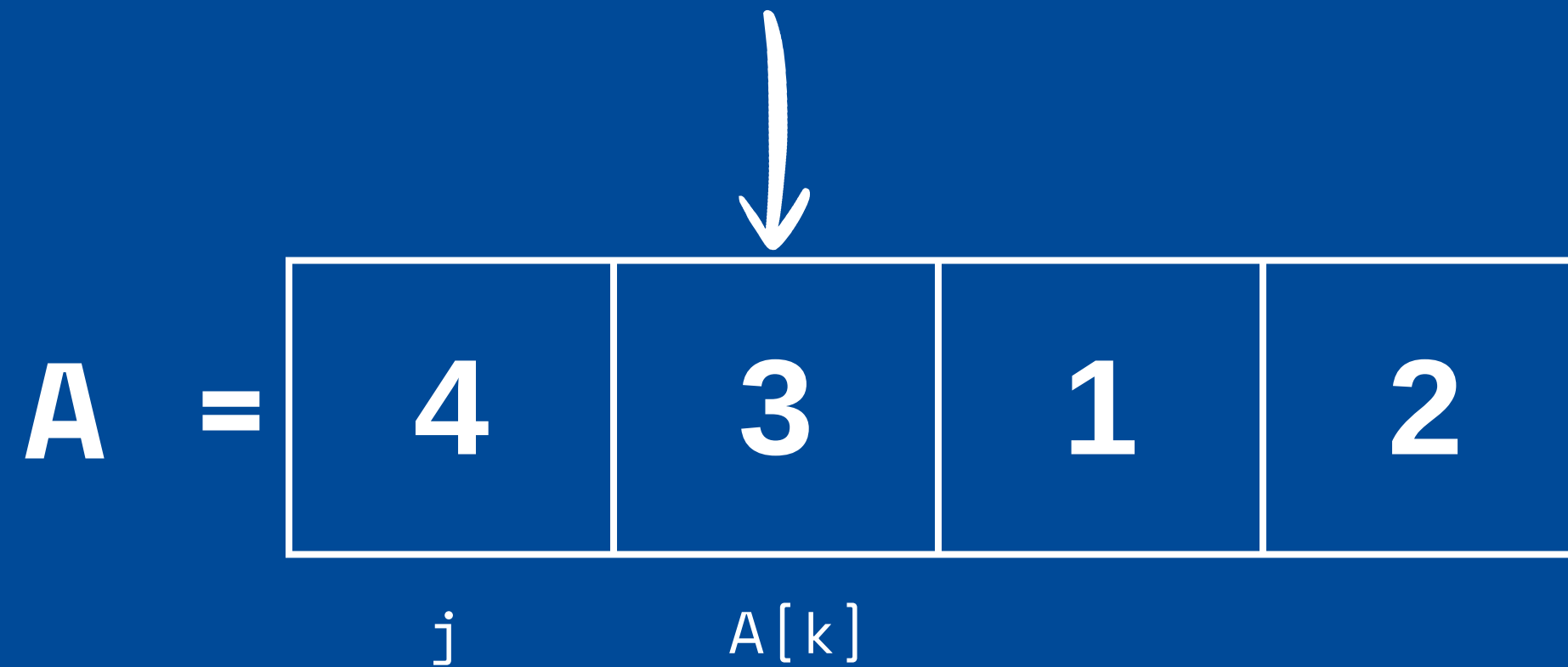
Vamos a recorrer con k

quick sort



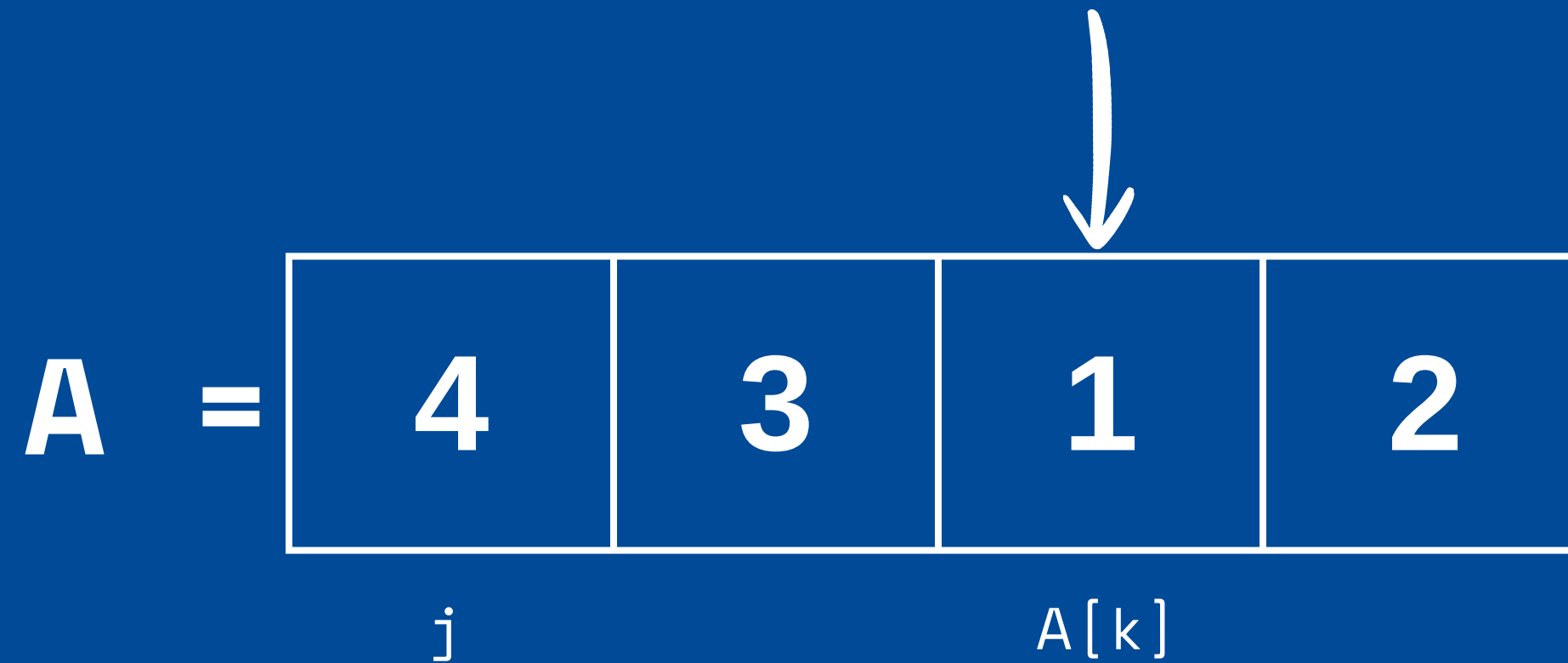
¿4 < 2? No, no hacemos nada

quick sort



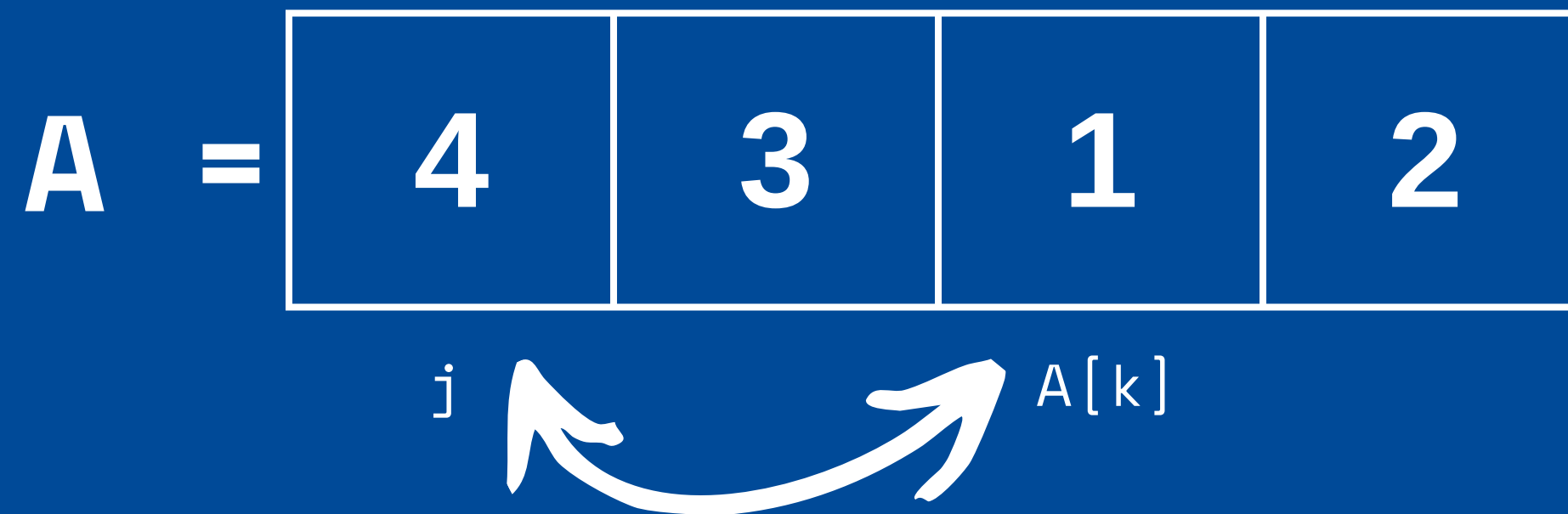
¿3 < 2? No, no hacemos nada

quick sort



$1 < 2$? Si

quick sort



$$A[j] \rightleftharpoons A[k] = A[0] \rightleftharpoons A[2]$$

quick sort

A =

1	3	4	2
----------	----------	----------	----------

j

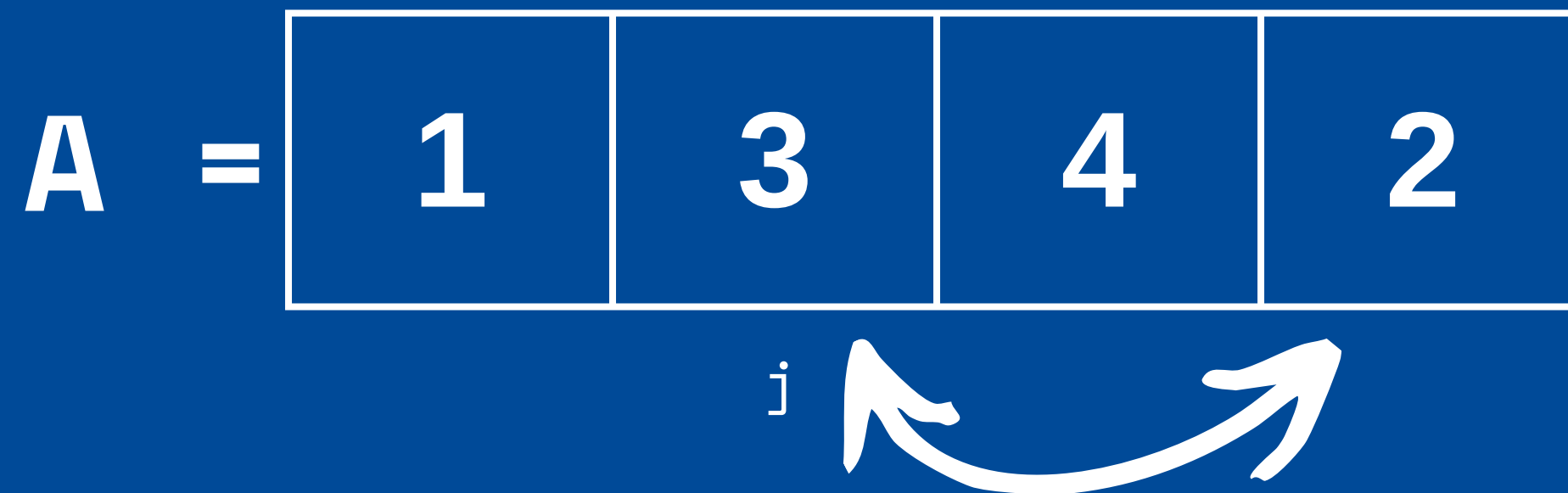
quick sort

A =

1	3	4	2
----------	----------	----------	----------

j

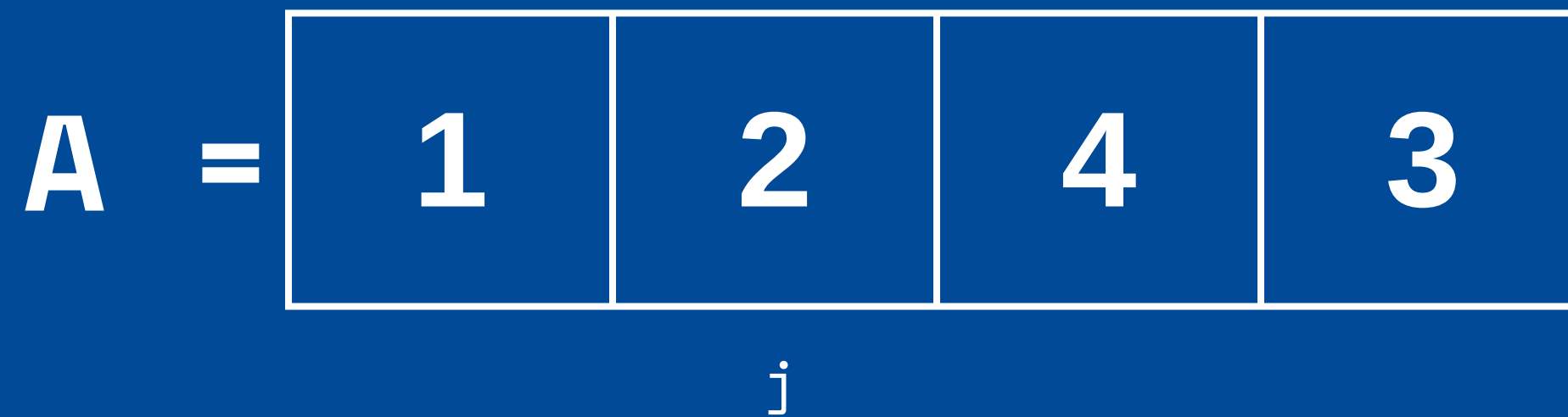
quick sort



Llevamos el pivote a su lugar

$$A[j] \rightleftharpoons A[f] = A[1] \rightleftharpoons A[3]$$

quick sort



2 quedó en su posición final

quick sort

A =

1	2	4	3
----------	----------	----------	----------

Siguientes llamados:

QuickSort(A, 0, 0) → termina
QuickSort(A, 2, 3) → seguimos

quick sort

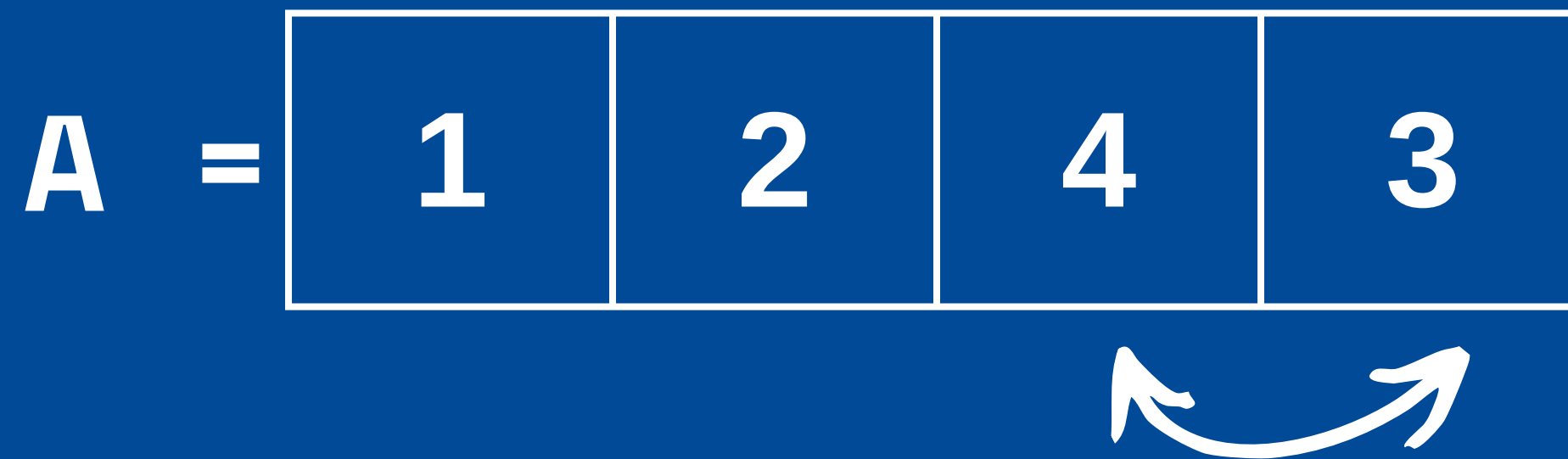


Elegimos un pivote aleatorio

$$x = 2$$

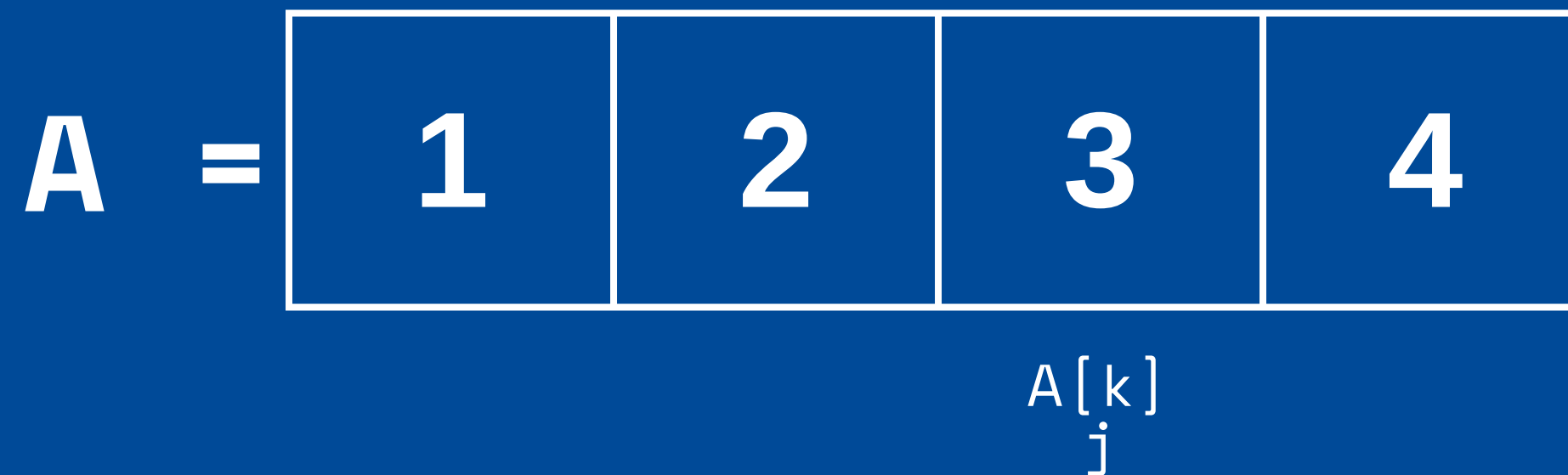
$$p = A[2] = 4$$

quick sort



Movemos el pivote al final

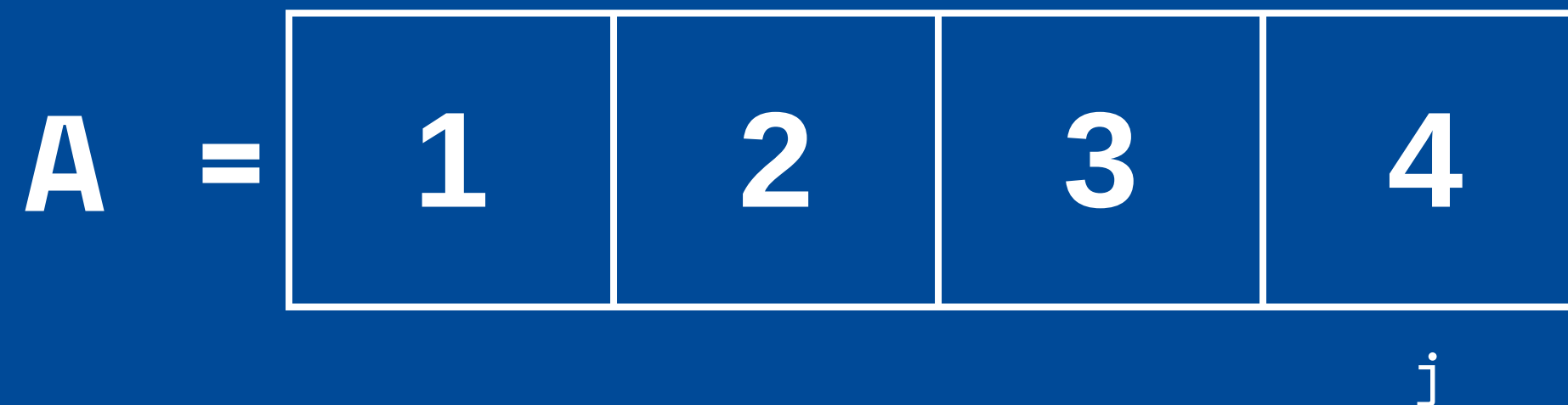
quick sort



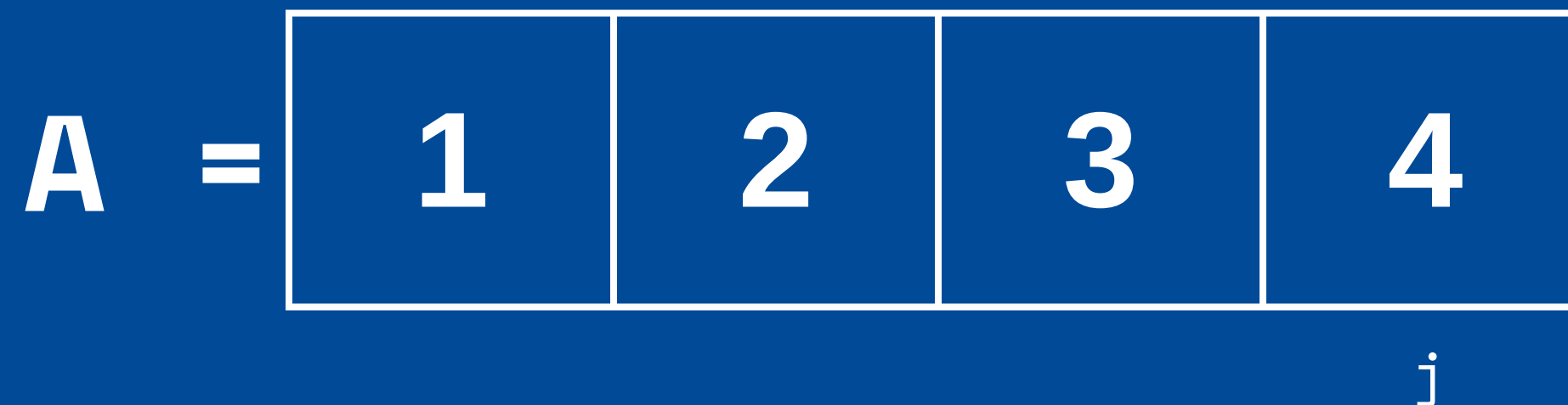
$3 < 4$? Si

Swap consigo mismo (porque $k = j$)

quick sort



quick sort



Llevamos el pivote a su lugar

$A[j] \rightleftharpoons A[f] = A[3] \rightleftharpoons A[3]$

(Ya está en su lugar)

Ejercicio 0



Los expertos en café te explican que la mejor forma de servir el cafe es la siguiente: Primero, revisar que el filtro esté bueno y hayan por lo menos 50g para moler. Después moler el café, y calentar el agua hasta los 94°C. Después se debe pasar el agua caliente disponible en la cafetera y pasarlo por los granos de café y después por un filtro.

Es importante no calentar de más el agua para que no se ponga amargo el café. Además, si se deja la moledora de granos de cafe prendida esta puede descomponerse, dejando la cafetera sin utilidad.

Escribe la función `preparar_cafe()`, la cual trabaja sobre una cafetera inteligente que posee las siguientes funciones:

Pregunta 0



prender()	Prende la cafetera y habilita el resto de funciones	Nulo
calentar_agua()	Comienza a calentar el agua	Nulo
temperatura()	Retorna la temperatura del agua.	Entero
moler_granos()	Muele una porción de los granos de café	Nulo

granos_restantes()	Retorna la cantidad de granos sin moler.	Entero
terminar(accion)	Recibe una acción como “calentar_agua” y finaliza la tarea	Nulo
estado_filtro()	Retorna el estado del filtro.	[malo, regular, bueno]
reemplazar_filtro()	Reemplaza el filtro de café por uno nuevo.	Nulo
servir_cafe()	Pasa agua caliente por los granos molidos y por un filtro, retorna el porcentaje de agua restante.	Float [0, 1]

Pregunta 0 - Solucion

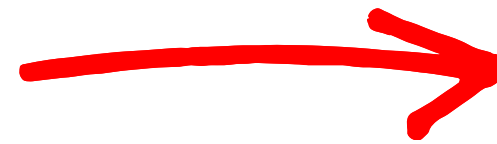


```
Preparar_cafe()
prender()
if estado_filtro()  $\neq$  Bueno then
    | reemplazar_filtro()
if granos_restantes() < 50 then
    | rellenar_granos(50-granos_restantes())
while granos_restantes() > 0 do
    | moler_granos()
calentar_agua()
while temperatura() < 94 do
    | continuar
terminar(calentar_agua)
restante = servir_cafe()
while restante > 0 do
    | restante = servir_cafe()
```

Pregunta 0 - Solucion



Esa función existía?
Nos consta que hace?
O cómo lo hace?



```
Preparar_cafe()
prender()
if estado_filtro()  $\neq$  Bueno then
  | reemplazar_filtro()
if granos_restantes() < 50 then
  | rellenar_granos(50-granos_restantes())
while granos_restantes() > 0 do
  | moler_granos()
calentar_agua()
while temperatura() < 94 do
  | continuar
terminar(calentar_agua)
restante = servir_cafe()
while restante > 0 do
  | restante = servir_cafe()
```

Pregunta 0 - Solucion



Dado que la forma en la que se rellenan los granos no influye en el objetivo de la pregunta basta con hacer algo de este estilo:

Aclaración:

- `rellenar_granos(cantidad)` es una función que recibe un número entero que representa la cantidad en gramos de granos de café que se van a rellenar.

Si nos pidiesen por ejemplo que el algoritmo sea $O(n)$:

- `rellenar_granos(cantidad) ...` (mismo texto que antes) + Esta función trabaja como la función $f(x)$ vista en clase que es $O(n)$

```
Preparar_cafe()
```

```
prender()
if estado_filtro() ≠ Bueno then
  | reemplazar_filtro()
if granos_restantes() < 50 then
  | rellenar_granos(50-granos_restantes())
while granos_restantes() > 0 do
  | moler_granos()
calentar_agua()
while temperatura() < 94 do
  | continuar
terminar(calentar_agua)
restante = servir_cafe()
while restante > 0 do
  | restante = servir_cafe()
```

Ejercicio 1



- a) Supongamos que al ejecutar Quicksort sobre un arreglo particular, la subrutina partition hace siempre el mayor número posible de intercambios; ¿cuánto tiempo toma Quicksort en este caso? ¿Qué fracción del mayor número posible de intercambios se harían en el mejor caso? Justifica

Pregunta 2 - I1-2020-2



Pregunta 2a - Solucion



- Intercambios de líneas 3 y 9 se hacen siempre
- Dentro del loop, sólo se hacen cuando $A[k] < p$
- Si queremos hacer la mayor cantidad de intercambios posibles, queremos que siempre $A[k] < p$.
- Entonces si tenemos un subarreglo de tamaño m , se hacen $2 + m - 1 = m + 1$ intercambios
- Quedan subarreglos de largos 0 y $m - 1$

Partition (A, i, f):

```
1   $x \leftarrow$  índice aleatorio en  $\{i, \dots, f\}$ 
2   $p \leftarrow A[x]$ 
3   $A[x] \rightleftharpoons A[f]$ 
4   $j \leftarrow i$ 
5  for  $k = i \dots f - 1$  :
6      if  $A[k] < p$  :
7           $A[j] \rightleftharpoons A[k]$ 
8           $j \leftarrow j + 1$ 
9   $A[j] \rightleftharpoons A[f]$ 
10 return  $j$ 
```

Pregunta 2a - Solucion



Con un arreglo de tamaño n tenemos:

$$\begin{aligned} T(n) &= n + (n - 1) + (n - 2) + \dots + 1 \\ &= \frac{n \times (n + 1)}{2} = \frac{1}{2} (n^2 + n) \\ &\in O(n^2) \end{aligned}$$



b) El algoritmo quicker-sort llama a la subrutina pq-partition, que utiliza dos pivotes p y q ($p < q$) para particionar el arreglo en 5 partes: los elementos menores que p , el pivote p , los elementos entre p y q , el pivote q , y los elementos mayores que q . Escribe el pseudocódigo de pq-partition. ¿Es quicker-sort más eficiente que quick-sort? Justifica.

Pregunta 2 - I1-2020-2



Pregunta 2b - Solucion



Ya que la definición dice que $p < q$, entonces es posible asumir que no hay datos repetidos. De todos modos, esto no afecta el análisis.

Lo más simple es implementar partition con listas ligadas como vimos en clases:

```
1: procedure PQ-PARTITION(lista ligada  $L$ )
2:    $p, q \leftarrow$  dos nodos de  $L$  tal que  $p < q$ . Quitar estos nodos de  $L$ .
3:    $A, B, C \leftarrow$  listas vacías  $\triangleright$  Los elementos menores a  $p$ , entre  $p$  y  $q$  y mayores a  $q$  respectivamente
4:   for nodo  $x \in L$  do
5:     if  $x < p$  then
6:       agregar  $x$  al final de  $A$ 
7:     else if  $x < q$  then
8:       agregar  $x$  al final de  $B$ 
9:     else
10:      agregar  $x$  al final de  $C$ 
11:    end if
12:  end for
13:  return  $A, p, B, q, C$ 
14: end procedure
```

El paso en la línea 2 es fácil de hacer en $\mathcal{O}(1)$, basta con extraer los primeros dos elementos de L , e intercambiarlos si $p > q$.

Pregunta 2b - Solucion



Recordemos que el peor caso de Quicksort se produce cuando partition separa una secuencia de m datos en dos secuencias disparejas, de 0 y $m - 1$ datos cada una. En ese caso la complejidad para una secuencia inicial de n datos es de:

$$T(n) = n + (n - 1) + (n - 2) + \dots + 1$$

$$T(n) \in \mathcal{O}(n^2)$$

En este caso esto también puede suceder, solo que se separa en 3 secuencias disparejas, de 0 , 0 y $m - 2$ elementos cada una. En este caso la complejidad para una secuencia inicial de n datos es de:

$$T(n) = n + (n - 2) + (n - 4) + \dots + 1$$

$$T(n) \in \mathcal{O}(n^2)$$

Ambos algoritmos son iguales en el peor caso, así que no podemos afirmar que quickersort sea mejor que quicksort.

Feedback

